Welcome

Digital and Computer Vision

Lesson 2: Session 1

Smoothing Techniques

Summary of the **previous** session: Image Basics & Manipulation

In the previous session we have covered :

Accessing pixel values.
Scaling, cropping, flipping and rotating images and translating images.
Perspective transformations and warping.
Changing Color-space (Grayscale, HSV, RGB).
Image arithmetic (adding, subtracting, blending images, bitwise operations).

Summary of the current session: Smoothing Techniques

This is the first session of our second lesson – Digital Image Processing.
In this session we will covered the following:

Blurring and smoothing techniques (Average, Gaussian, Median, Bilateral filters).
Removing noise using image filtering.
Histogram equalization for contrast enhancement.
Exercise: image smoothing and noise reduction.

Exercise : from previous session

Find an algorithm to extract blue object, from messi5.jpg

Algorithm to Extract a Specific Color :

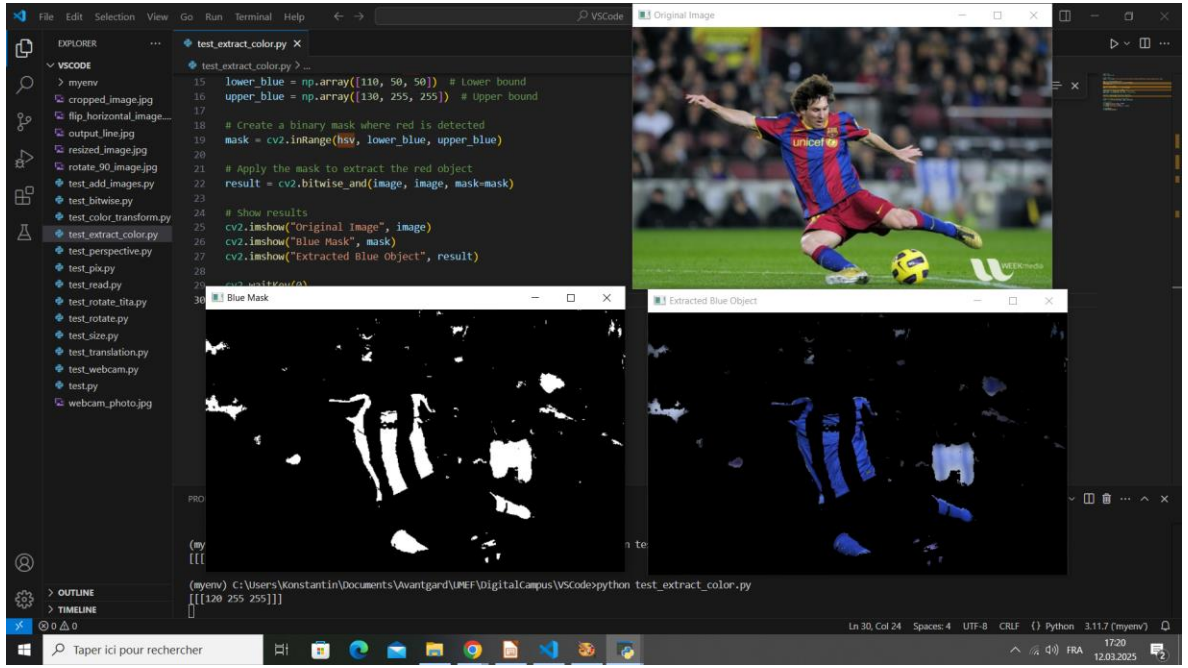
1. Convert the image from BGR to HSV (Hue, Saturation, Value).
2. Define a color range (lower and upper HSV values).
3. Use cv2.inRange() to create a binary mask.
4. Apply cv2.bitwise_and() to keep only the desired color.

Exercise : complete code 1/2

```
import cv2
import numpy as np

image = cv2.imread("../Images/messi5.jpg")

# Find the HSV blue color
blue = np.uint8([[[255,0,0 ]]])
hsv_blue = cv2.cvtColor(blue, cv2.COLOR_BGR2HSV)
print( hsv_blue )

# Convert BGR to HSV
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# Define color range for RED (HSV format)
lower_blue = np.array([110, 50, 50])  # Lower bound
upper_blue = np.array([130, 255, 255])  # Upper bound
```

Exercise : complete code 1/2

```
# Create a binary mask where red is detected
mask = cv2.inRange(hsv, lower_blue, upper_blue)

# Apply the mask to extract the red object
result = cv2.bitwise_and(image, image, mask=mask)

# Show results
cv2.imshow("Original Image", image)
cv2.imshow("Blue Mask", mask)
cv2.imshow("Extracted Blue Object", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



Extract blue object from video stream

Let's now change a bit this exercise in order to track in real time blue object from a video stream. We will do this directly in VS Code editor.

2D Convolution (Image Filtering)

Images also can be filtered with various filters which helps in removing noise, blurring images, or in finding edges.

The most widely used type of neighborhood operator is a linear filter, where an output pixel's value is a weighted sum of pixel values within a small neighborhood

45	60	98	127	132	133	137	133
46	65	98	123	126	128	131	133
47	65	96	115	119	123	135	137
47	63	91	107	113	122	138	134
50	59	80	97	110	123	133	134
49	53	68	83	97	113	128	133
50	50	58	70	84	102	116	126
50	50	52	58	69	86	101	120

$f(x,y)$

*

0.1	0.1	0.1
0.1	0.2	0.1
0.1	0.1	0.1

$h(x,y)$

=

69	95	116	125	129	132
68	92	110	120	126	132
66	86	104	114	124	132
62	78	94	108	120	129
57	69	83	98	112	124
53	60	71	85	100	114

$g(x,y)$

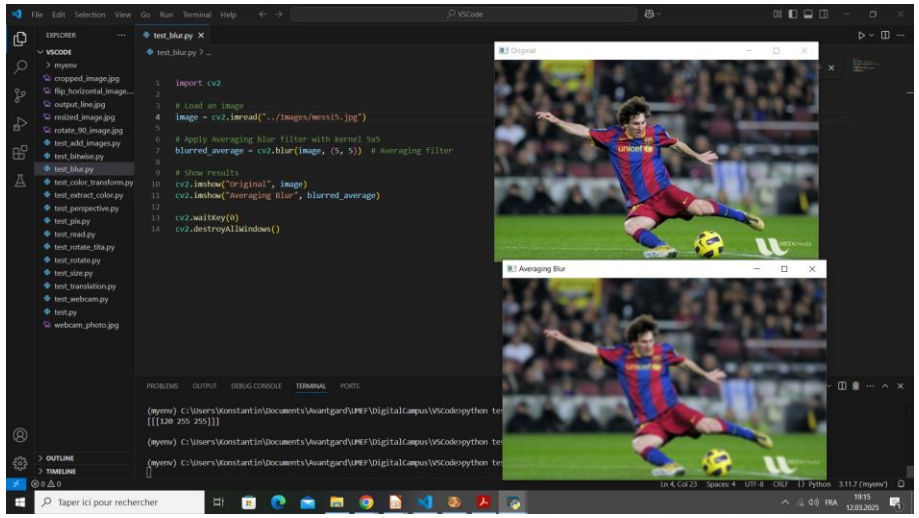
Szeliski, R. "Computer Vision: Algorithms and Applications." (https://szeliski.org/Book/)

The matrix on the left is convolved with the filter in the middle to yield the matrix on the right. The light blue pixels indicate the source neighborhood for the light green destination pixel.

Averaging blur example

Average blur simply takes the average of all the pixels under the kernel area and replaces the central element. This is done by the function cv.blur()

```
import cv2

image = cv2.imread("../Images/messi5.jpg")

# Apply Averaging blur filter with kernel 5x5
blurred_average = cv2.blur(image, (5, 5))  # Averaging filter

# Show results
cv2.imshow("Original", image)
cv2.imshow("Averaging Blur", blurred_average)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



Gaussian Blurring

Unlike simple averaging, Gaussian blur applies a weighted filter, giving more importance to pixels near the center. The sum of the kernel values is normalized to 1 (so brightness remains unchanged). A 3×3 Gaussian kernel might look like this:

$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

The result is a smooth and natural blur, commonly used in preprocessing, noise reduction and edge detection. In OpenCV for Python use :

cv2.GaussianBlur(src, (ksize_x, ksize_y), sigmaX)

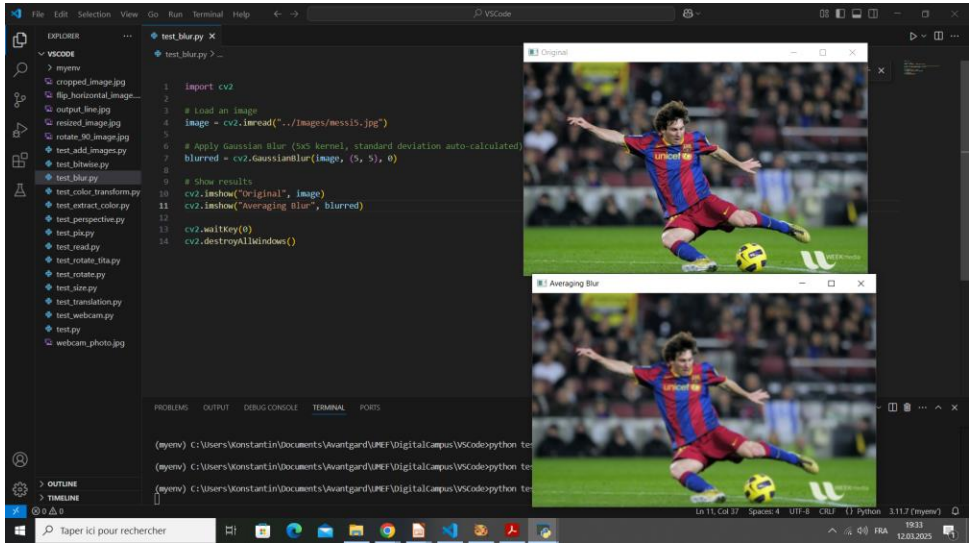
src → Input image.

(ksize_x, ksize_y) → Kernel size (should be odd numbers, like (3,3), (5,5)).

sigmaX → Standard deviation. If 0, OpenCV automatically calculates a suitable value.

Gaussian Blurring example

```
import cv2

# Load an image
image = cv2.imread("../Images/messi5.jpg")

# Apply Gaussian Blur (5x5 kernel, standard deviation auto-calculated)
blurred = cv2.GaussianBlur(image, (5, 5), 0)

# Show results
cv2.imshow("Original", image)
cv2.imshow("Averaging Blur", blurred)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



Median Blurring

Median blurring takes the median of all the pixels under the kernel area and the central element is replaced with this median value. This removes extreme noise while keeping edges sharp.

[3, 9, 1, 12, 17, 4, 57]
[1, 3, 4, 9, 12, 17, 57]
Median is : 9

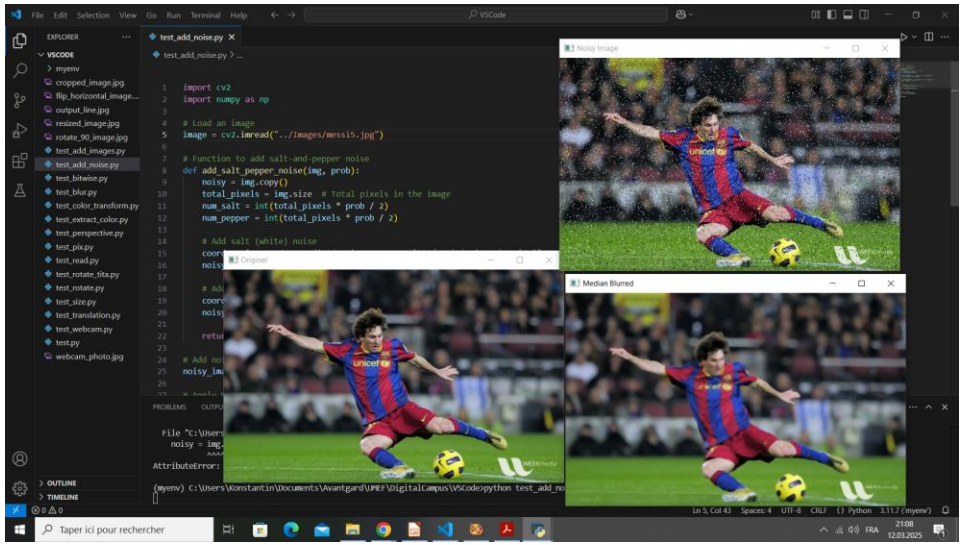
It is highly effective against salt-and-pepper noise in an image.

Median Blurring example 1/2

```
import cv2
import numpy as np

image = cv2.imread("../Images/messi5.jpg")
# Function to add salt-and-pepper noise input the probability to be noise pixel
def add_salt_pepper_noise(img, prob):
    noisy = img.copy()
    total_pixels = img.size  # Total pixels in the image
    num_salt = int(total_pixels * prob / 2) # White pixels in the noisy image
    num_pepper = int(total_pixels * prob / 2) # Black pixels in the noisy image
   # Add salt (white) noise
    coords = [np.random.randint(0, i - 1, num_salt) for i in img.shape[:2]]
    noisy[coords[0], coords[1]] = [255, 255, 255]
    # Add pepper (black) noise
    coords = [np.random.randint(0, i - 1, num_pepper) for i in img.shape[:2]]
    noisy[coords[0], coords[1]] = [0, 0, 0]

    return noisy
```

Median Blurring example 2/2

```
# Add noise to the image
noisy_image = add_salt_pepper_noise(image, 0.02)  # 2% noise

# Apply Median Blurring (kernel size = 5)
blurred_image = cv2.medianBlur(noisy_image, 5)

# Show images
cv2.imshow("Original", image)
cv2.imshow("Noisy Image", noisy_image)
cv2.imshow("Median Blurred", blurred_image)

cv2.waitKey(0)
cv2.destroyAllWindows()
```



Bilateral filters

Bilateral Filtering is a powerful image smoothing technique that reduces noise while preserving edges. Unlike Gaussian and Median blurring, which smoothen everything (including edges), bilateral filtering only blurs similar colors while keeping sharp transitions intact.

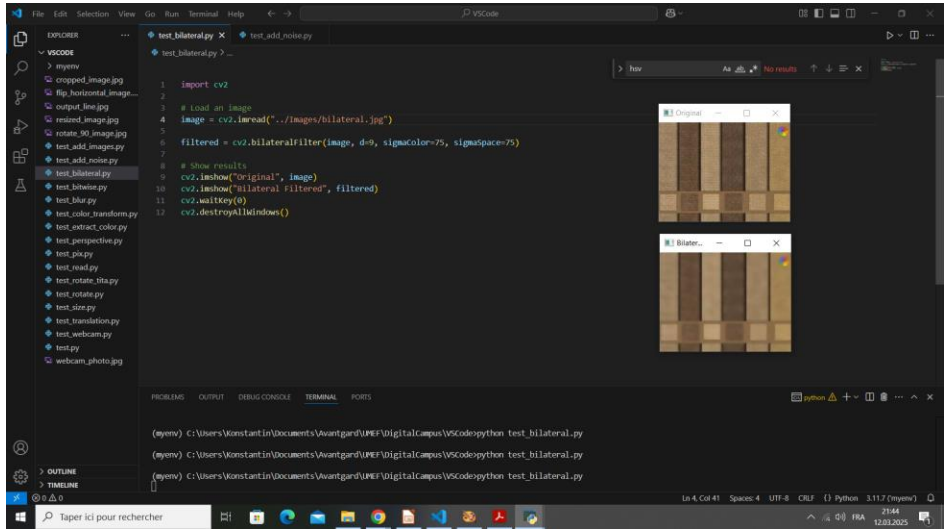
How Bilateral Filtering Works :
1. Each pixel is replaced by a weighted average of its neighbors.
2. Two factors determine the weight of each neighbor:
Spatial distance (closer pixels have more influence).
Color intensity similarity (similar colors contribute more, different colors contribute less).

This prevents edges from getting blurred because pixels with different colors have much lower influence.

In OpenCV we use the function cv2.bilateralFilter(src, d, sigmaColor, sigmaSpace)
src → Input image.
d → Diameter of the pixel neighborhood (higher = stronger effect).
sigmaColor → How much colors influence blurring (higher = more smoothing).
sigmaSpace → How far pixels influence each other (higher = wider area).

Bilateral filtering example

```
import cv2

# Load an image
image = cv2.imread("../Images/bilateral.jpg")

filtered = cv2.bilateralFilter(image, d=9, sigmaColor=75, sigmaSpace=75)

# Show results
cv2.imshow("Original", image)
cv2.imshow("Bilateral Filtered", filtered)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Summary of noise reduction techniques

Image noise refers to random variations in color or brightness that distort an image. It can be caused by:
Low light conditions (sensor noise)
Transmission errors (compression artifacts)
Environmental interference (dust, motion blur)

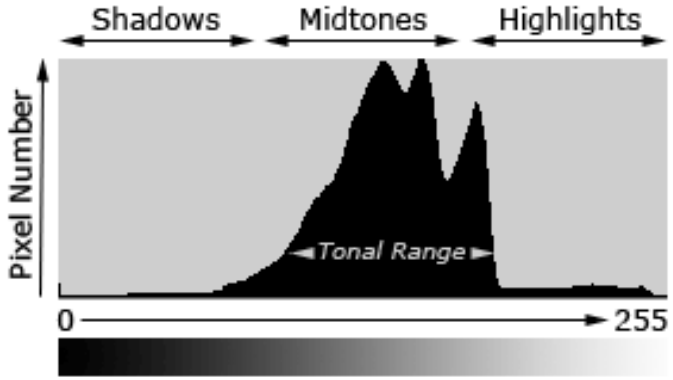
To improve image quality, image filtering techniques are used to reduce noise while preserving important details.

Filter	Best for	Drawback
Averaging	General noise removal	Blurs edges
Gaussian Blur	Softening noise while keeping detail	Reduces sharpness
Median Blur	Removing salt-and-pepper noise	Slower on large images
Bilateral Filter	Noise reduction while keeping edges sharp	Computationally expensive

Histograms

A histogram is a graphical representation of the distribution of pixel intensities in an image. It shows how many pixels have a specific intensity value, helping us analyze brightness, contrast, and exposure.

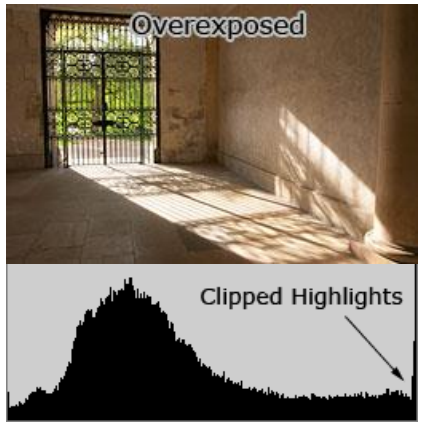
Understanding Image Histograms :
X-axis (Pixel Intensity) → Ranges from 0 (black) to 255 (white) (for grayscale images).
Y-axis (Frequency) → Shows the number of pixels with that intensity.

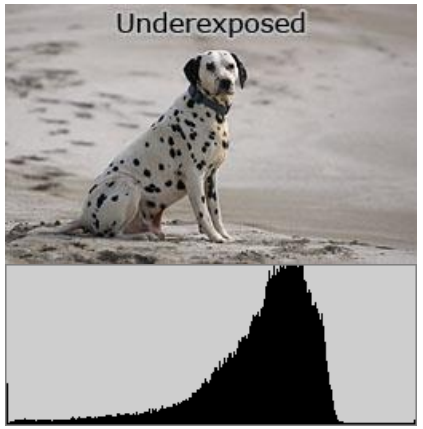


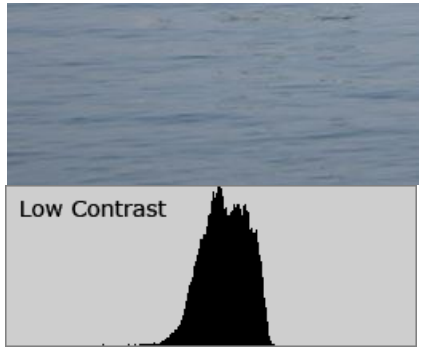
https://www.cambridgeincolour.com/tutorials/histograms1.htm

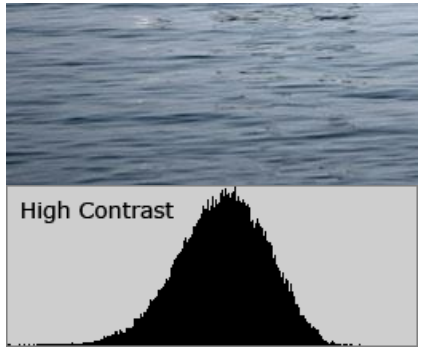
Histograms for Image Brightness & Contrast

Dark images → Histogram peaks on the left.
Bright images → Histogram peaks on the right.
High contrast images → Histogram spread across the entire range.









https://www.cambridgeincolour.com/tutorials/histograms1.htm

Why Use Histograms in Image Processing?

Analyze Image Brightness & Contrast
Enhance Images (Histogram Equalization)
Thresholding & Segmentation
Object Detection (Skin Color Filtering, Background Removal)

Histogram Calculation in OpenCV

In OpenCV we use cv.calcHist(images, channels, mask, histSize, ranges[, hist[, accumulate]]) function to calculate histogram
images : it is the source image of type uint8 or float32. it should be given in square brackets, ie, "[img]".
channels : it is also given in square brackets. It is the index of channel for which we calculate histogram.
mask : mask image. To find histogram of particular region of image, you have to create a mask
histSize : this represents our BIN count. Need to be given in square brackets. For full scale, we pass [256].
ranges : this is our RANGE. Normally, it is [0,256].

Numpy also provides you a function, np.histogram(). However OpenCV function is faster (around 40X). So stick with OpenCV function.

Plotting Histograms with Matplotlib

matplotlib.pyplot is a module in Matplotlib, a popular Python library used for plotting and visualizing data. It provides an easy-to-use interface similar to MATLAB's plotting functions.

Function	Description
`plt.plot(x, y)`	Line plot
`plt.bar(x, y)`	Bar chart
`plt.scatter(x, y)`	Scatter plot
`plt.hist(data, bins=10)`	Histogram
`plt.imshow(image, cmap='gray')`	Display an image
`plt.xlabel("Label")`	X-axis label
`plt.ylabel("Label")`	Y-axis label
`plt.title("Title")`	Plot title
`plt.legend()`	Show legend
`plt.show()`	Display the plot

Commonly Used pyplot Functions

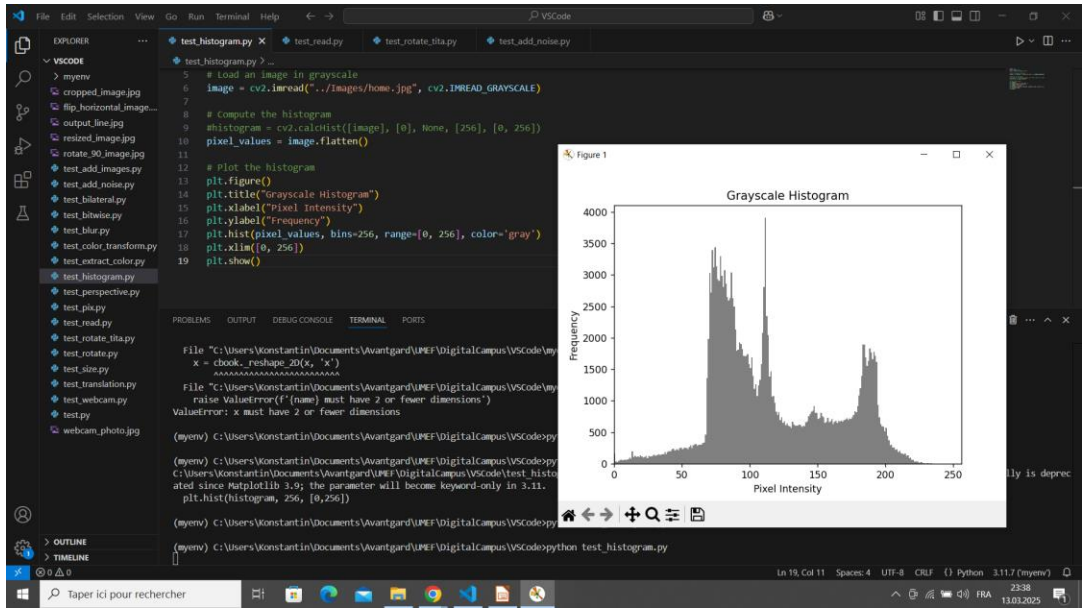
Instal matplotlib library

In the command prompt type :

pip install matplotlib

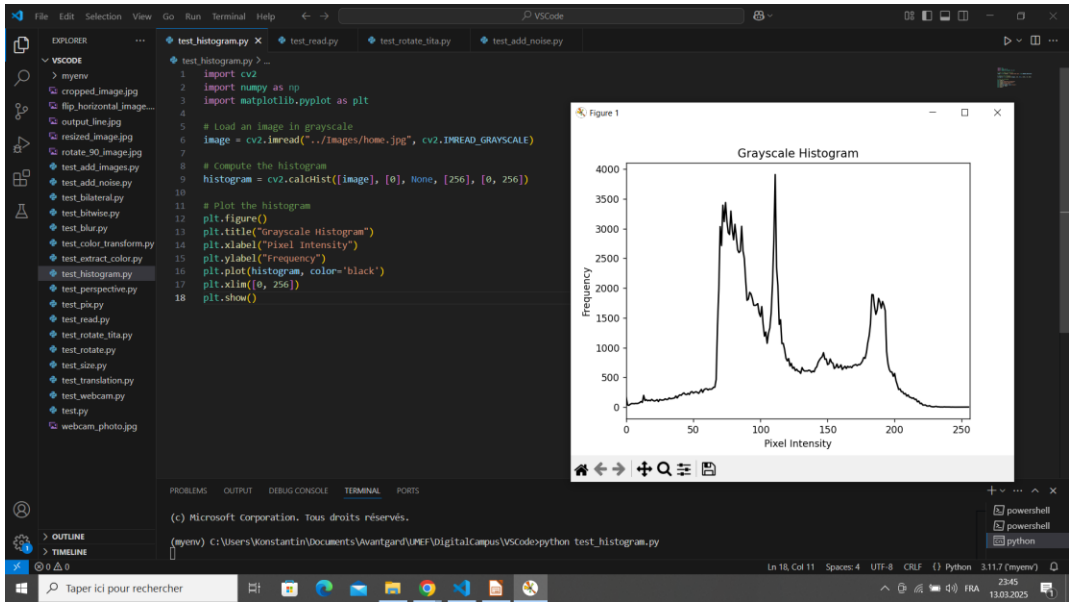
Histograms in greyscale example

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load an image in grayscale
image = cv2.imread("../Images/home.jpg", cv2.IMREAD_GRAYSCALE)

# Compute the histogram
pixel_values = image.flatten()

# Plot the histogram
plt.figure() # plot new figure, canvas
plt.title("Grayscale Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.hist(pixel_values, bins=256, range=[0, 256], color='gray')
plt.xlim([0, 256]) # limit x axis
plt.show()
```



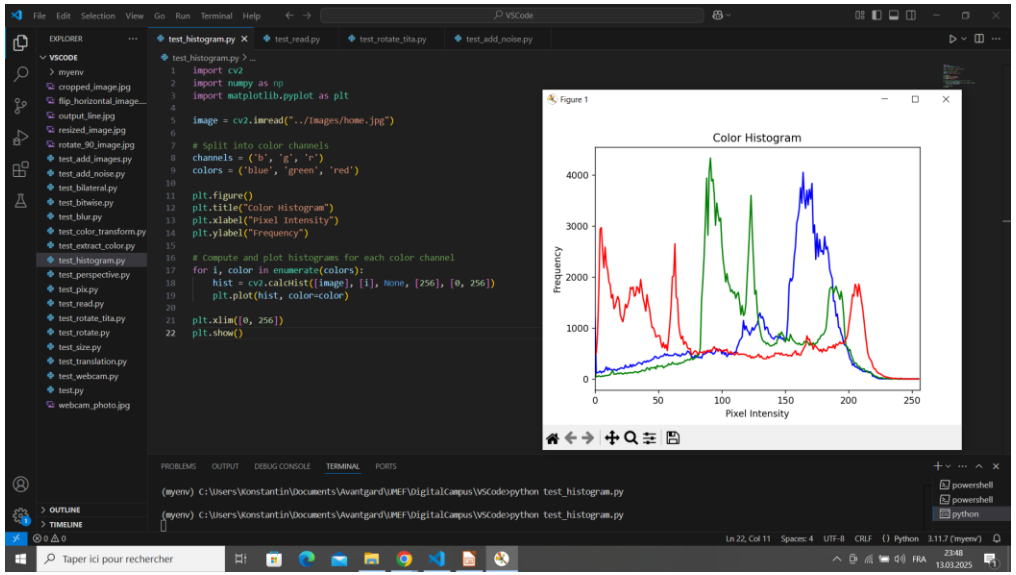
Histograms in greyscale as a line plot

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load an image in grayscale
image = cv2.imread("../Images/home.jpg", cv2.IMREAD_GRAYSCALE)

# Compute the histogram
histogram = cv2.calcHist([image], [0], None, [256], [0, 256])

# Plot the histogram
plt.figure()
plt.title("Grayscale Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.plot(histogram, color='black')
plt.xlim([0, 256])
plt.show()
```



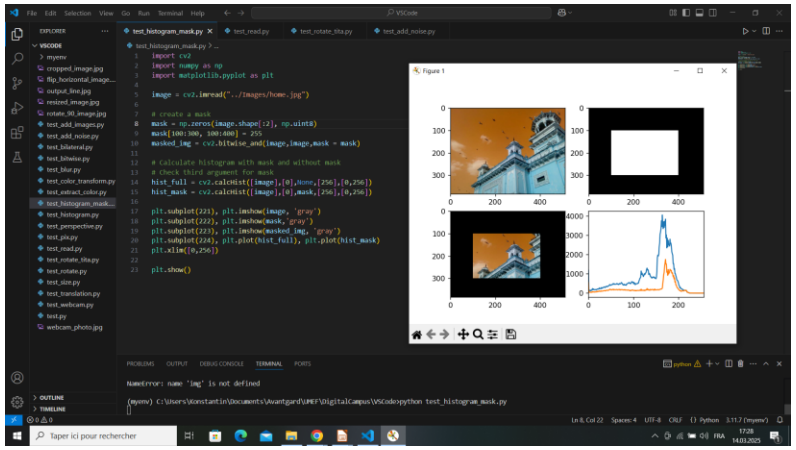
Example Code for Color Histogram

```
image = cv2.imread("../Images/home.jpg")

# Split into color channels
channels = ('b', 'g', 'r')
colors = ('blue', 'green', 'red')

plt.figure()
plt.title("Color Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

# Compute and plot histograms for each color channel
for i, color in enumerate(colors):
    hist = cv2.calcHist([image], [i], None, [256], [0, 256])
    plt.plot(hist, color=color)

plt.xlim([0, 256])
plt.show()
```



Example Application of Mask

```
# create a mask
mask = np.zeros(image.shape[:2], np.uint8)
mask[100:300, 100:400] = 255
masked_img = cv2.bitwise_and(image,image,mask = mask)

# Calculate histogram with mask and without mask
hist_full = cv2.calcHist([image],[0],None,[256],[0,256])
hist_mask = cv2.calcHist([image],[0],mask,[256],[0,256])
 plt.subplot(221), plt.imshow(image, 'gray')
plt.subplot(222), plt.imshow(mask,'gray')
plt.subplot(223), plt.imshow(masked_img, 'gray')
plt.subplot(224), plt.plot(hist_full), plt.plot(hist_mask)
```



Histogram Equalization for Contrast Improvement

Histogram equalization is a technique in image processing used to enhance contrast by redistributing the pixel intensity values more evenly across the image.

Algorithm :

1. Calculate the histogram of the image (distribution of pixel intensities).
2. Compute the cumulative distribution function (CDF).
3. Normalize the CDF to spread intensity values across the full range (0 to 255).
4. Map the New Pixel Intensities: Apply the transformation to the original image, adjusting pixel intensities accordingly. The CDF acts like a lookup table to transform pixel intensities from the original distribution to a new, more balanced one.

For RGB images, equalization is applied individually to each channel in HSV color space (to avoid color distortion).

Histogram Equalization - equations

✅ **CDF Formula:**

$$CDF(i) = \sum_{j=0}^{i} P(j) \tag{1}$$

Where:

- $P(j)$ is the **probability of each intensity value**.
- $CDF(i)$ is the cumulative sum up to intensity i.

The CDF is then scaled to **spread pixel values between 0 and 255**:

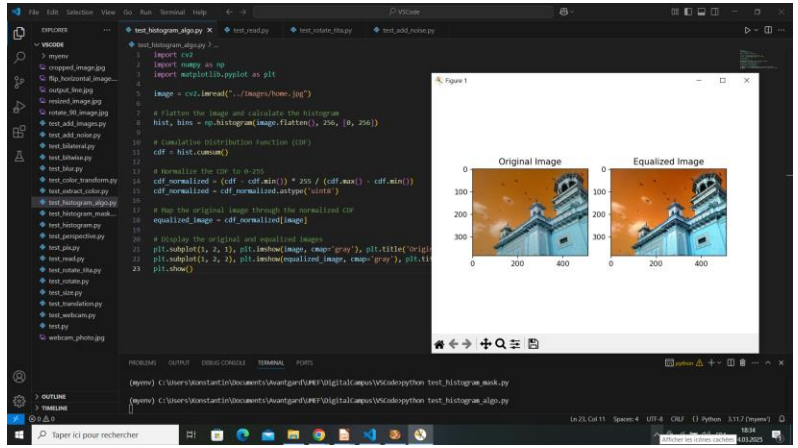
$$C' = \frac{CDF(i) - CDF_{\min}}{(M \times N) - CDF_{\min}} \times (L - 1) \tag{2}$$

Where:

- C' is the **new pixel intensity**.
- $M \times N$ is the total number of pixels.
- L is the number of intensity levels (256 for grayscale images).
- $CDF_{\min}$ is the **minimum non-zero value of the CDF**, which prevents stretching the darkest areas too much.

Histogram Equalization algorithm’s code

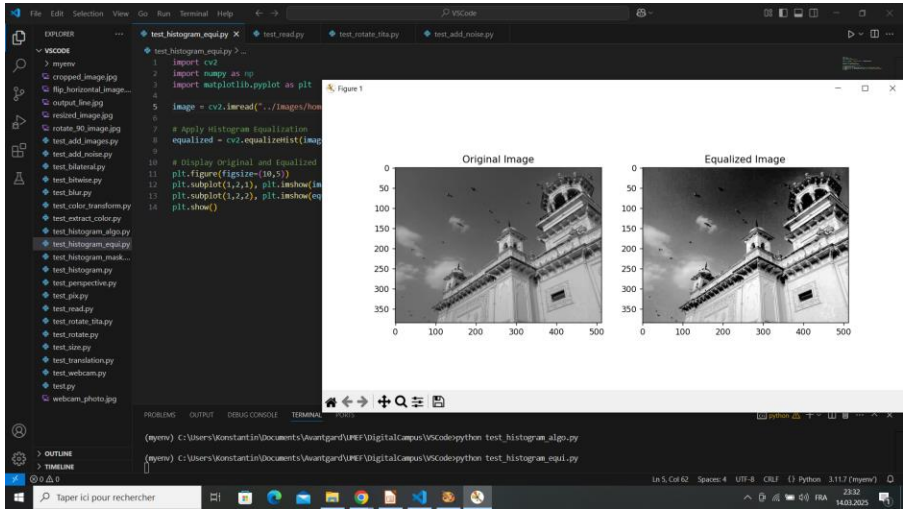
```
image = cv2.imread("../Images/home.jpg")
# Flatten the image and calculate the histogram
hist, bins = np.histogram(image.flatten(), 256, [0, 256])
# Cumulative Distribution Function (CDF)
cdf = hist.cumsum()

# Normalize the CDF to 0-255
cdf_normalized = (cdf - cdf.min()) * 255 / (cdf.max() - cdf.min())
cdf_normalized = cdf_normalized.astype('uint8')

# Map the original image through the normalized CDF
equalized_image = cdf_normalized[image]

# Display the original and equalized images
plt.subplot(1, 2, 1), plt.imshow(image, cmap='gray'), plt.title('Original Image')
plt.subplot(1, 2, 2), plt.imshow(equalized_image, cmap='gray'), plt.title('Equalized Image')
plt.show()
```



Example: Histogram Equalization in OpenCV

In OpenCV this algorithm is applied in finction :
cv2.equalizeHist(image)
Which equalizes the histogram of a grayscale image.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

image = cv2.imread("../Images/home.jpg", cv2.IMREAD_GRAYSCALE)

# Apply Histogram Equalization
equalized = cv2.equalizeHist(image)

# Display Original and Equalized Images
plt.figure(figsize=(10,5))
plt.subplot(1,2,1), plt.imshow(image, cmap='gray'), plt.title("Original Image")
plt.subplot(1,2,2), plt.imshow(equalized, cmap='gray'), plt.title("Equalized Image")
plt.show()
```



Example: Applying Histogram Equalization to Color Images

```
# Load the color image
image = cv2.imread("image.jpg")

# Convert to HSV color space
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# Equalize only the V (Brightness) channel
hsv[:,:,2] = cv2.equalizeHist(hsv[:,:,2])

# Convert back to BGR color space
equalized_color = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

# Display result
cv2.imshow("Original", image)
cv2.imshow("Equalized Color", equalized_color)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

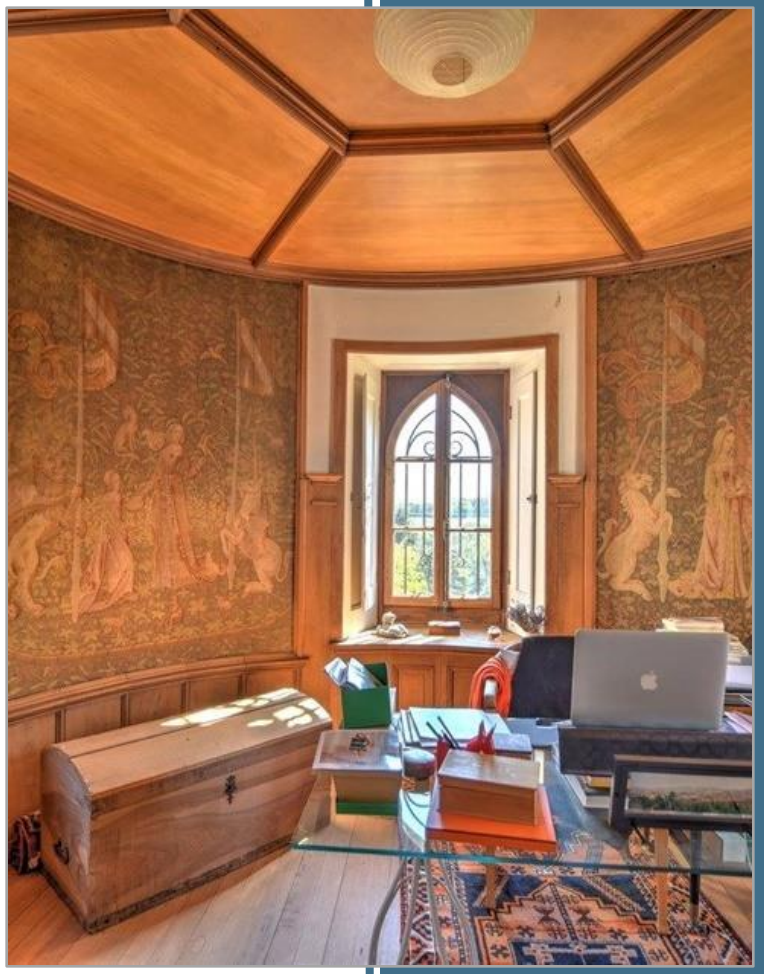
Exercises for this session

Apply and explain histogram equalization with the method CLAHE (Contrast Limited Adaptive Histogram Equalization) of OpenCV

In this session we have covered:

Blurring and smoothing techniques (Average, Gaussian, Median, Bilateral filters).
Removing noise using image filtering.
Histogram equalization for contrast enhancement.
Exercise: image smoothing and noise reduction.

In the next session we will covered:

Thresholding , Edge Detection & Contours
Detecting edges using Sobel, Laplacian, and Canny edge detection.
Finding and drawing contours in images.
Convex hulls and shape approximations.

THANKS !!!